

Informe del proyecto

1. Comparar ambos ordenamientos

Antes de entrar en números, hay un detalle que conviene aclarar: la tabla de Los puntajes guardan como máximo 10 registros. Con tan poquitos datos los dos algoritmos terminan al instante, así que compararlos usando la tabla real no sirve de nada, gana el que sea pero en menos de un parpadeo. Para que la comparación tenga sentido hay que probar con cantidades de datos cada vez más grandes, que es justo lo que hace la pantalla de mejores puntajes del juego: mide los dos algoritmos sobre 2000 registros y deja repetir la medición.

Para este informe hice lo mismo pero con varios tamaños: 100, 250, 500, 1000, 2000, 4000, 8000, 16000, 32000 y 64000 datos, todos con puntajes al azar. Sobre cómo medir: el reloj de la máquina no es muy fino (no baja de un milisegundo), así que repito cada ordenamiento varias veces hasta juntar unos 40 ms y promedie. También le reste el tiempo de copiar el arreglo, para quedarme solo con el tiempo de ordenar, y de tres corridas me quede con la mas rapida para bajar el ruido de otros programas que andaban corriendo. Todo lo compile en modo release; en modo debug los tiempos salen mas grandes, pero la forma de las curvas es la misma.

Estos fueron los resultados:

n	Inserción	Quicksort	Veces más lento	ins/n^2	$quick/(n \cdot \log n)$
100	0.009	0.005	1.9	0.85	6.80
250	0.031	0.010	3.2	0.50	4.90
500	0.123	0.029	4.2	0.49	6.54
1000	0.625	0.070	8.9	0.63	7.06
2000	2.125	0.188	11.3	0.53	8.55
4000	9.625	0.414	23.2	0.60	8.65
8000	32.000	0.922	34.7	0.50	8.89
16000	135.000	2.094	64.5	0.53	9.37
32000	583.000	4.563	127.8	0.57	9.53
64000	2328.000	9.625	241.9	0.57	9.42

(los tiempos están en milisegundos; las dos últimas columnas están multiplicadas por un millón para que se puedan leer, su valor exacto da igual, lo que importa es que se mantengan estables)

Coincide con lo que dice la teoría? Si, y bastante bien. La idea es simple: si un algoritmo tarda mas o menos n^2 , entonces al dividir su tiempo entre n^2 tiene que quedar siempre un número parecido, sin importar cuán grande sea n . Y eso es exactamente lo que pasa con la inserción: la columna ins/n^2 se queda entre 0.5 y 0.85 prácticamente de principio a fin. Si fuera $n \cdot \log n$, ese número habría ido bajando a medida que crecen los datos, y no pasa.

Con el quicksort es al revés: su tiempo dividido entre $n \cdot \log n$ se mantiene acotado (más o menos entre 7 y 9.5), que es la señal de un $n \cdot \log n$.

Lo que más me convenció fue mirar cuantas veces es más lento uno que el otro. Ese número pasa de 1.9 a 241.9, y cada vez que duplicó n se multiplica por casi 2 (64.5, después 127.8, después 241.9). Si uno compara las dos fórmulas, la razón debería crecer como $n/\log n$, y justamente al duplicar n esa expresión se multiplica por un factor cercano a 2. Los datos siguen esa predicción.

¿A partir de qué tamaño se nota? Depende de qué entendamos por "notar". Como proporción, ya con 500 o 1000 datos la inserción es entre 9 y 16 veces más lenta. Pero como tiempo que uno sienta de verdad, la inserción recién empieza a pasar los 10 ms alrededor de los 4000 datos, se pone molesta como a los 16000 (135 ms) y ya a los 64000 es un desastre: 2.3 segundos contra 9.6 milisegundos del quicksort. En resumen, para la tabla real (máximo 10) la diferencia no existe; a partir de unos miles de registros se empieza a ver, y de decenas de miles en adelante es enorme. Tal cual lo que dice la notación asintótica.

Una salvedad que vale mencionar: el quicksort es rápido en promedio. Con datos casi ordenados y este pivote del medio, sin protecciones, se puede degradar a n^2 . La comparación de arriba usa datos al azar, que es su caso normal.

2. POR QUE EL TABLERO ES UNA LISTA DE FILAS

El tablero está armado como una lista enlazada de 20 nodos, uno por fila, y cada El nodo guarda sus 10 celdas en un arreglito. O sea, la estructura fuerte (cuantas filas hay y en que orden estan) es la lista; el contenido de cada fila es cosa aparte.

Esto viene muy bien justo para limpiar líneas. Pensemos que hay que hacer cuando se completa una fila: borrarla, meter una fila vacía arriba y dejar que todo lo de arriba baje un puesto. Con una lista, eso se traduce casi todo a mover punteros:

- Borrar la fila es desenganchar su nodo.
- Que las filas de arriba "bajen" sale gratis, porque el orden de las filas lo decide el encadenamiento, no donde están los datos. Las celdas de las filas que quedan ni se tocan.
- Meter la fila vacía arriba es enganchar un nodo al principio.

Si el tablero fuera un arreglo de dos dimensiones, borrar una fila obligaría a copiar todas las celdas de las filas de arriba para bajarlas, o a rehacer el arreglo entero. Bastante más trabajo.

Los costos concretos en mi implementación. La función que limpia hace una sola pasada por la lista y arma las cosas al mismo tiempo: por un lado va enlazando las filas que se quedan, y por otro va juntando las filas completas, a las que

primero les borra las 10 celdas y después reutiliza como las filas nuevas de arriba. Entonces:

- Meter una fila vacía al frente cuesta 2 operaciones (apuntar el nuevo nodo a la cabeza vieja y mover la cabeza). Es constante.
- Borrar las filas completas y reponer las vacías cuesta una pasada por las 20 filas, más lo que se tarda en vaciar las celdas de las filas borradas. En total es del orden de $\text{alto} + k \cdot \text{ancho}$, donde k son las filas borradas (a lo sumo 4). Nada más. Las filas que sobreviven no se mueven.

Comparado con el arreglo, que costaría del orden de $\text{alto} \cdot \text{ancho}$ copias de celdas por cada limpieza, la lista hace bastante menos trabajo.

Lo que se paga a cambio: buscar una fila por su número ya no es inmediato. La función que devuelve la fila N recorre la lista desde el principio, así que cuesta alto en el peor caso, y como casi todo el código del tablero usa esa función (por ejemplo para revisar choques), esas operaciones también se encarecen un poco. Con 20 filas no se nota para nada, pero es el precio honesto de este diseño: se gana un montón al meter y sacar filas, y se pierde en el acceso directo a una fila cualquiera.

3. POR QUE EL REPLAY NECESITA LISTA DOBLE

El replay guarda cada movimiento como un nodo que tiene, además del estado del juego en ese momento, dos punteros: uno al movimiento anterior y otro al siguiente. La lista lleva aparte el primero, el último y un puntero "actual" que marca en qué paso estamos (cuando es nulo, estamos al inicio de todo).

La razón de necesitar los dos punteros es que el replay se maneja para los dos lados: uno puede ir retrocediendo y avanzando por la partida, y además puede deshacer y rehacer mientras juega. Eso obliga a poder caminar la línea de tiempo en ambos sentidos.

Con una lista simple (solo puntero al siguiente) uno podría avanzar sin problema, pero para retroceder un solo paso tendría que recorrer la lista desde el principio hasta encontrar el nodo anterior al actual, y eso cuesta tanto como movimientos haya. Para colmo, cuando uno retrocede y hace un movimiento nuevo hay que cortar la rama de "rehacer" que quedaba; con doble puntero ese corte es directo, mientras que con lista simple habría que buscar el nodo previo, otra vez recorriendo todo.

Sobre los costos, entonces: retroceder un paso y avanzar un paso son inmediatos, solo hay que leer el puntero que corresponde y copiar el estado guardado. Registrar un movimiento nuevo también es inmediato para engancharlo al final, más el costo de borrar la rama de rehacer si existía, que es proporcional a los movimientos que se descartan. Conviene aclarar que copiar el estado no es gratis del todo, porque cada estado incluye el tablero entero (20 por 10), la cola de

hasta 32 piezas y algunas banderas, pero es un tamaño fijo que no depende de que tan larga ha sido la partida, así que no cambia el análisis.

En una frase: la lista doble permite ir para adelante y para atrás en tiempo constante, y una simple obligaría a recorrer toda la partida para retroceder o para cortar una rama.

4. LA COLA DE EVENTOS Y POR QUE EL FRENTE SIEMPRE ES EL PRÓXIMO

La cola de eventos es una lista enlazada que se mantiene ordenada por el momento en que cada evento debe dispararse, de menor a mayor. Cuando se programa un evento nuevo, la función se encarga de insertarlo en el lugar que le corresponde: si es el más temprano de todos (o la lista está vacía) lo pone de primero; si no, avanza hasta el último evento cuyo tiempo sea menor o igual al nuevo y lo mete justo ahí.

Gracias a eso, la lista nunca se desordena. Como las únicas otras operaciones que existen son sacar el del frente y preguntar si el del frente ya toca, el invariante se mantiene solo. Y si la lista está ordenada, el primero de todos es siempre el de menor tiempo, o sea el próximo a dispararse. No hay que buscar nada, se mira la cabeza y listo. Un detalle es que la comparación usa "menor o igual", así que si dos eventos caen en el mismo instante el nuevo queda detrás, respetando el orden en que llegaron.

Cuanto cuesta insertar: si el evento es el más temprano, es inmediato (se pone al frente). Si es el más tardío, hay que recorrer toda la lista hasta el final, así que el peor caso es proporcional a la cantidad de eventos que haya. En general cuesta recorrer los eventos que tengan tiempo menor o igual al nuevo.

En el juego esto no se siente para nada, porque la agenda se arma una sola vez con 36 eventos y durante la partida no se agregan más. Eso sí, como esos 36 se programan en orden creciente, cada uno se inserta al final y termina costando un recorrido completo, lo que da un total del orden de n^2 para armar toda la agenda. Con 36 eventos son unos mil y pico de comparaciones, algo que no se nota. Si algún día la agenda fuera mucho más grande, valdría la pena guardar también un puntero al final de la lista para insertar en tiempo constante en este caso, que es el más común, o usar un montículo. Se eligió la lista ordenada porque da acceso inmediato al evento que sigue, que es lo que más se consulta, y deja el código sencillo.